Code Map — Where Things Are

What Each Folder Does

Models (`models/`) Defines the shape of the data stored in MongoDB. Each model is a schema that describes what fields a document has, what type they are, and any rules (required, min, max etc.). The model is what you use to read and write to the database. Example: `models/user.js` defines what a user document looks like in the database.

Controllers (`controllers/`) Handles incoming HTTP requests from the frontend. A controller function receives the request, does some checks, calls services or the database, and sends back a response. Controllers are the entry point for the API. Example: `controllers/auth.controller.js` handles POST /api/auth/login — it checks the password, and sends back a token.

Services (`services/`) Contains the business logic — the actual "thinking" of the application. Controllers are kept simple by moving complex logic into services. Services don't know about HTTP requests or responses, they just do a job and return a result. Example: `services/auth.services.js` handles token generation and rotation. `services/game.services.js` handles ELO calculations and dice rolling.

Middleware (`middleware/`) Code that runs between the request arriving and the controller handling it. Used for things like checking if the user is logged in, or validating the request body before it reaches the controller. Example: the auth middleware checks the access token on protected routes. The validate middleware checks that the request data is valid.

Routers (`routers/`) Maps URLs to controller functions. Defines which HTTP method and path triggers which controller, and which middleware runs first. Example: `routers/routes/game.routes.js` maps POST /api/games to the `createRoom` controller.

Utils (`utils/`) Small reusable helper functions that don't belong to any specific feature. Things like hashing passwords, signing JWTs, or managing timers. Example: `utils/hash.js` has the hashPwd and checkPwd functions. `utils/jwt.js` has the signAccessToken function.

WebSocket (`websocket/`) Handles real-time communication using Socket.io. Unlike HTTP which is request/response, WebSocket keeps a connection open so the server can push updates to the client instantly. Used for live game events like rolling dice and placing bets. Example: `websocket/gameSocket.js` listens for events like `roll_dice` and `place_bet` from players.

Auth

What	Where
Register, login, logout, refresh, email verification endpoints	`controllers/auth.controller.js`
Token generation, token rotation, email verification logic	`services/auth.services.js`
Password hashing function	`utils/hash.js`

What	Where
Password hashing hook (when it runs)	`models/user.js` — `pre("validate")`
Refresh token sent as httpOnly cookie	`controllers/auth.controller.js` — `login` and `refresh`
Refresh token cleared on logout (server-side)	`services/auth.services.js` — `revokeRefreshToken`
Refresh token cookie cleared on logout (client-side)	`controllers/auth.controller.js` — `logout`
Cookie security options (httpOnly, secure, sameSite)	`controllers/auth.controller.js` — `_refreshTokenCookieOptions`
Email verification token generation	`services/auth.services.js` — `generateVerificationToken`

User Model

What	Where
User schema (all fields)	`models/user.js`
Auto-generated userId	`models/user.js` — `pre("validate")`
Fields hidden from queries by default (select: false)	`models/user.js` — `pwd`, `refreshToken`, `emailVerificationToken`, `emailVerificationExpiry`
Fields removed before sending to client	`models/user.js` — `toJSON` transform

Game

What	Where
Game schema (all fields)	`models/game.js`
Auto-generated gameId	`models/game.js` — `pre("validate")`
Create a game room	`controllers/game.controller.js` — `createRoom`
Join a game room	`controllers/game.controller.js` — `joinRoom`
Leave a game room	`controllers/game.controller.js` — `leaveRoom`
Check user has enough points before joining	`controllers/game.controller.js` — `createRoom` and `joinRoom`
Deduct buy-in points when joining	`controllers/game.controller.js` — `joinRoom` and `createRoom`
Return buy-in points when leaving	`controllers/game.controller.js` — `leaveRoom`

What	Where
Game starts automatically when room is full	`controllers/game.controller.js` — `joinRoom`
Get all games (paginated, filterable)	`controllers/game.controller.js` — `getAllGames`
Get a single game	`controllers/game.controller.js` — `getGame`
Filter out other players' dice until reveal	`services/game.services.js` — `filterRollsForUser`
Add username and ELO to player data in responses	`services/game.services.js` — `enrichGameWithUserInfo`

Game Logic (Services)

What	Where
Poker dice faces defined	`services/game.services.js` — `POKER_FACES`
Roll 5 dice	`services/game.services.js` — `rollDice`
Determine round/game winner	`services/game.services.js` — `determineWinners`
Return remaining points to player profiles after game	`services/game.services.js` — `returnPointsToProfiles`
ELO rating update after game	`services/game.services.js` — `updateELORatings`
ELO calculation formula	`services/game.services.js` — `calculateNewELO`

WebSocket (Real-time)

What	Where
All real-time game events	`websocket/gameSocket.js`
Player joins a game socket room	`websocket/gameSocket.js` — `join_game` event
Player rolls dice	`websocket/gameSocket.js` — `roll_dice` event
Player ends turn early	`websocket/gameSocket.js` — `end_turn` event
Player holds dice (position shared, not values)	`websocket/gameSocket.js` — `hold_dice` event
Player places a bet	`websocket/gameSocket.js` — `place_bet` event
Player disconnects — 30 second grace period	`websocket/gameSocket.js` — `disconnect` event
Player marked as abandoned after disconnect	`websocket/gameSocket.js` — `disconnect` event
Reconnecting player gets their game state restored	`websocket/gameSocket.js` — `sendCurrentRollToReconnectingPlayer`